Sentiment Analysis-Based Study on Chinese Audiences Acceptance of "Friends"

Group 8 STAT 2630SEF

Introduction

Friends is a long-running sitcom that has attracted a large global audience, but cultural differences, such as plot developments, determine its acceptance in China.

This study uses a model to analyze online comments and explore the acceptance of Friends among Chinese audiences. By analyzing the emotions conveyed in the comments and key dialogue themes and phrases, it reveals some of the attitudes of Chinese viewers towards Friends.

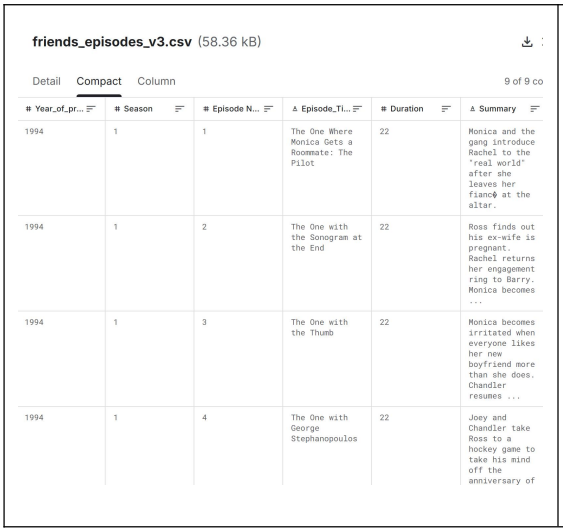
Method

We haved devided the whole into 4 steps, which are Data collect, Data Extract, Make Model, and Data Virtualization.

1. Data Collect (ODS->DWS)

1.1. Data Source

We use douban platform as our data source. Douban as most popular and television review community in China. We through this platform to scrapy chinese users short comments and rating. And then, in order to compare with the rating between China and America, we get some csv files from Kaggle.

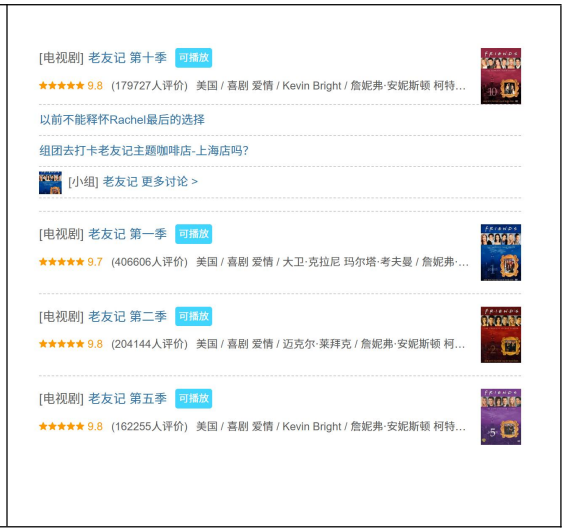


friends_episodes_v3.csv (58.36 kB)

Detail Compact Column 9 of 9 co

Year_of_pr...	Season	Episode N...	Episode_Ti...	Duration	Summary
1994	1	1	The One Where Monica Gets a Roommate: The Pilot	22	Monica and the gang introduce Rachel to the "real world" after she leaves her fianc◆ at the altar.
1994	1	2	The One with the Sonogram at the End	22	Ross finds out his ex-wife is pregnant. Rachel returns her engagement ring to Barry. Monica becomes ...
1994	1	3	The One with the Thumb	22	Monica becomes irritated when everyone likes her new boyfriend more than she does. Chandler resumes ...
1994	1	4	The One with George Stephanopoulos	22	Joey and Chandler take Ross to a hockey game to take his mind off the anniversary of

Kaggle data file



[电视剧] 老友记 第十季 可播放

★★★★★ 9.8 (179727人评价) 美国 / 喜剧 爱情 / Kevin Bright / 詹妮弗·安妮斯顿 柯特...

以前不能释怀Rachel最后的选择

组团去打卡老友记主题咖啡店-上海店吗?

[小组] 老友记 更多讨论 >

[电视剧] 老友记 第一季 可播放

★★★★★ 9.7 (406606人评价) 美国 / 喜剧 爱情 / 大卫·克拉尼 玛尔塔·考夫曼 / 詹妮弗·...

[电视剧] 老友记 第二季 可播放

★★★★★ 9.8 (204144人评价) 美国 / 喜剧 爱情 / 迈克尔·莱拜克 / 詹妮弗·安妮斯顿 柯...

[电视剧] 老友记 第五季 可播放

★★★★★ 9.8 (162255人评价) 美国 / 喜剧 爱情 / Kevin Bright / 詹妮弗·安妮斯顿 柯特...

Douban platform

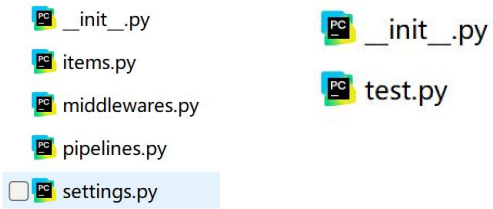
1.2. Acquisition Technology

We use Python scrapy to collect data from Douban platform. We used Python Scrapy as our crawling framework. The basic process of Python Scrapy have 4 steps.

Operation code	Examples of pictures

scrapy startproject+"the name of scrapy file"	`C:\Users\任钱\Desktop>scrapy startproject douban`
	`New Scrapy project 'douban', using template directory`
	`ckages\scrapy\templates\project', created in:`
	`C:\Users\任钱\Desktop\douban`
	
	`You can start your first spider with:`
	`cd douban`
	`scrapy genspider example example.com`

scrapy genspider+"scrapy name"+"crawler website"	`C:\Users\任钱\Desktop\douban>scrapy genspider test www.douban.com`
	`Created spider 'test' using template 'basic' in module:`
	`douban.spiders.test`

Python file edit	
scrapy crawl+"scrapy name"	`C:\Users\任钱\Desktop\douban>scrapy crawl test -o data.csv`

There also have more some detail about Python Scrapy.

Regarding 'Python file edit', the different Python file have different function. There we only talk some

basic files.

1) item.py: This file is used to set data schema. It defined data schema and data type so that provide uniform structural standard for data collect, data cleaning and storage.

2) settings.py: This file is used to set project configuration.

3) test.py: This is core crawler implementation file. You need to edit code to achieve crawl data from website to the specified file.

And then talk about scrapy output command.

scrapy crawl+"scrapy name": Look at picture, the '-o data.csv' represent that file name and the type of file. If you want to output json file, you can 'scrapy crawl test -o filename.json'.

1.3.Data Schema & Storage

1) Data Schema

About the data of douban platform, we design 2 tables, a 'ods_douban_comment table' and 'ods_douban_season_rating'. So they have different data schema.

Table 1: ods-douban-comment Fields: id season (int) comment (String) name (String)	Table 2: ods _ douban _ season _ rating Fields: season (string) rating (double) total _ people (int)

ods_douban _comment table ods_douban_season_rating table

Regarding the data that come from Kaggle, since they are already well organized and public accessible, so we preserved the orginal data structure and import data into a table that called 'ods_friends_episodes_v3'.

Table3: ods _ friends _ episodes _ v3
Fields: Year-of-prod (int)
Season (int)
Episode Number (int)
Episode _ Title (String)
Summary (String)
Director (String)
Rating (double)
Votes (int)

ods_friends_episodes_v3 table

2) Data Storage

We stored the collected data in the MongoDB-based ODS database. The full name of ODS is 'Operational Data Store'. This layer is used for temporary storage layer and construct a connection

between raw data sources and data warehouse.

1.4.Data Size

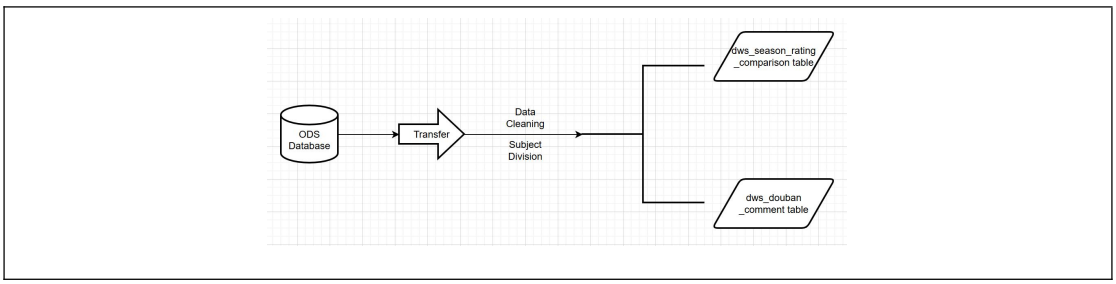
We use Python Scrapy to scrapy 4000 short comments from douban ,then output as CSV file. And we get 10 rating data of chinese audiences. Also we get 10 rating data of American audiences.

2. Data Extract

2.1. Process

Focus on Data Extract, we followed the path that from ODS to ETL process, and eventually to the DWS layer.

The full name of ETL is 'Extract, Transform, Laod'. The process that combines data from mutiple data sources into a single. In our project, we extracted some important data and loaded into MongoDB-based DWS database. We built this database through topic aggregation and data cleaning based on the ODS layer data.



The workflow of Data Extract

2.2. Detail

1) Data cleaning

In this project, during the ETL process from the ODS to the DWS layer, we first perform data deduplication, followed by sorting the records in ascending order based on the TV series season.

2) Subject divison

Based on the field topic of ODS layer, we divide and summarize the data into two core topic tables: rating comparison table and comment table. The first table aggregates rating indicators of each season to quantify audience feedback, and the second table aggregates audience comments of each season. After data cleaning and unifying the indicator caliber, we finally form a standardized and efficient indicator query service in DWS layer, providing solid data support for modeling and analyzing the acceptance of friends in China in ads layer.

3) Some code

In order to improve the code readability and data search effeciency, we use some sql statement.

```
#生成 dws_season_rating_comparison表
df_dws_rating=duckdb.query("""
  SELECT d.season,
      d.rating AS df_douban_rating,
      i.imdb_rating,
      ROUND((d.rating-i.imdb_rating),1) AS rating_difference
  FROM df_ods_douban_rating d
  LEFT JOIN (SELECT ROUND(AVG(Stars),1) AS imdb_rating,
             Season
         FROM df_ods_imdb_rating
         GROUP BY Season) i
  ON d.season=i.Season
  ORDER BY d.season
""").df()
```

```
#生成 dws_douban_comment
df_dws_comment=duckdb.query("""
  SELECT id AS user_id,
      season,
      "name" AS user_name,
      comment
  FROM df_ods_comment
  ORDER BY season
""").df()
```

2.3. Subject Table Construction

We constructed 2 subject table: a 'dws_season_rating_comparison' table and a 'dws_ douban_comment' table. They have different function.

1) dws_season_rating_comparison: Aggregates rating indicators of each season to quantify audience feedback.

2) dws_ douban_comment: Aggregates audience comments of each season.

Table 1. dws _ season _ rating _ comparison Fields: season (int) douban-rating (double) imdb-rating (double) rating -difference (double)	Table 2: dws _ douban _ comment Fields: user _ id name (String) comment (String) season (int)

dws_season_rating_comparison table　　dws_douban_comment table

3. Make Model

Make Model in this project, is the core analysis model. And it is called ADS. ADS is the Application Data Store, which enable data to quickly serve specific application scenarios in a business-friendly form. It has 7 steps to make sure that the source data is clean, and put the raw comment to the accurate, structured, standardized results:

DWS Comment Raw Data
STEP1: Rule-Based Filtering
STEP2: Model-Based Filtering
STEP3: Text Processing
STEP4: Sentiment Analysis
STEP5: Topic Classification
STEP6: Merge Sentiment And Topic Results
STEP7: Output Results

*I-M-3 1　Resume Model Overall Seven-Step Flowchart

3.1 Rule-Based Filtering

This steps is used to filter obvious junk data with low-cost rules to reduce the cost of subsequent model filtering calculation.

It contain 4 steps, which are Null value handling, Text standardization, Content filtering, and

Deduplication.

Null Value Handling will check if the comment content is empty, if it is empty, delete the entire comment, including any lines it contains. Text Standardization will remove all the backspace in the comment. Content Filtering will remove the comment if the comment just contain symbols or number, after finish all of it , it will get the comment whilch it length is less than 2, and remove it from the list. Deduplication will move the comment that with the same content. Follow four rules:

Comment	Season	User Name	Operation
S	S	S	Delete ONE of them
S	S	NS	Delete ONE of them
S	NS	NS	Delete ALL
NS	S	S	Keep the LONGEST comment
*(S: Same, NS: Not Same)			

*I-M-3-1 1 Strategies for dealing with duplicate comments

After all these steps, it will output a new table called dws_comment_rule_filtered, the raw comments number is 4235, and output comments becomes 3781.

3.2 Model-Based Filtering

This step use 4 Models to filtering the garbage comment. Which are:

Model Name	Purpose
Titeiiko/OTIS-Official-Spam-Model	Pre-filter
textdetox/xlmr-large-toxicity-classifier-v2	Toxicity /Aggressiveness Detection
mrm8488/bert-tiny-finetuned-sms-spam-detection	Advertising / Junk Detection
baptistejamin/xlm-roberta-large-spam_v4	Multi-class garbage detection

*I-M-3-2 1 Model use in 3.2 and purpose

The first model is use the pre-filter the comments, if a comment was injured by the garbage labels left

by this model, this comment will be remove from the next step. Because this model has a relatively lenient judgment standard, if a comment is labeled as spam by this model, then there is a 99.5% probability that this comments is indeed a spam comment.

The second model is a aggressiveness detection model, which it will come up with some aggressive or negative comments. After that, it will output two tag with this confidence level.

The third model is a spam detection model, which it will detect the spam comment, and the ouput of it just like the second model.

The fourth model is a three classification model, and it will classify spam, gibberish, and regular comment.

The Score fomula of the second and third model is :

$$\text{Score}_n = \begin{cases} CE, & \text{if Tag is Regular} \\ 1 - CE, & \text{otherwise} \end{cases}$$
*CE: Coefficient of Noral tag

*I-M-3-2 2 Score fomula of the model

The final score formula is:

$$\text{Score}_{\text{final}} = \text{Score}_{\text{sec}} \times 0.2 + \text{Score}_{\text{tir}} \times 0.35 + \text{Score}_{\text{fou}} \times 0.45$$

*I-M-3-2 3 Final fomula of comment

Only if the final score bigger or equal than 0.6 can get into the next step.

The weighting strategy used in it was developed based on multiple experiments:

Model 2 despite a little criticism, whether it is offensive or not, it will be judged as an aggressive label. Even when someone drops a pretty mild critique, like saying "There's no other good character in this drama," it automatically slaps an "aggressive" label on it. That's obviously not ideal, so we dialed its weight way down.

Model 3 is actually solid at catching spam, but here's the catch: it has a 30-40% false negative rate, meaning it frequently nukes comments we actually want to keep. Kind of a problem, so we landed on 0.35 for this one.

Model 4 is our three-class classifier and honestly performs better than the other two across most scenarios. That said, it's not infallible—it still makes mistakes. So we gave it the highest weight, but

with a safeguard: if the other models push back, Model 4 can't just steamroll through and get approved anyway. Keeps things balanced.

After the model was cleaned up, the number of comments changed from 3781 to 2642 after the rule was cleaned up.

3.3 Text Pre-Processing

So here's how we clean up the text before feeding it into the models. First, we split sentences based on ending punctuation—periods, question marks, that kind of stuff—but we keep commas intact because we don't want to wreck the internal structure of each sentence.

Next, we run everything through jieba for tokenization, then strip out stopwords using the standard stopwords library. Any sentence that ends up with fewer than 2 valid tokens after all this gets tossed out. Finally, we glue the remaining sentences back together with periods as separators.

3.4 Sentiment Analysis And Keyword Extraction

This is the core step in ADS, we through it to get the sentiment and get the keyword. In this step, we use three models and one LLM arbitration based on Deepseek.

Type	Model	Remarks
Local Model	nlptown/bert-base-multilingual-uncased-sentiment	Sentiment
Local Model	cardiffnlp/twitter-xlm-roberta-base-sentiment	Sentiment
LLM	deepseek/deepseek-chat	Arbitration
Local Model	paraphrase-multilingual-MiniLM-L12-v2	Keyword

*I-M-3-4 1 The type of model use with its name and remarks in 3.4

3.4.1 Sentiment Analysis

The first model will output a 5-star scale, the score of the comment it will get is:

$$P(\text{sentiment}|\text{star}=s)=\begin{cases}\mathbf{N}=1,\mathbf{Ne}=0,\mathbf{P}=0 & s=1\\ \mathbf{N}=0.7,\mathbf{Ne}=0.3,\mathbf{P}=0 & s=2\\ \mathbf{N}=0,\mathbf{Ne}=1,\mathbf{P}=0 & s=3\\ \mathbf{N}=0,\mathbf{Ne}=0.3,\mathbf{P}=0.7 & s=4\\ \mathbf{N}=0,\mathbf{Ne}=0,\mathbf{P}=1 & s=5\end{cases}$$

*I-M-3-4-1 2 5 Star model score mapping

The second model is special, since this model cannot accept too long sentences, we will use the result after processing 3.3. It will output the coefficient of Negative, Neutral, and Positive of each sentence, and then we normalization. After output we will get all the length of each sentences, through these length, we can get the each ratio of sentence length, and then we perform a weighted summation, get the result coeficient of this comment.

The single sentences that model will output is:

$$\mathbf{p}^{(i)}=\begin{bmatrix}p_{\mathbf{N}}^{(i)}\\ p_{\mathbf{Ne}}^{(i)}\\ p_{\mathbf{P}}^{(i)}\end{bmatrix},\quad \text{s.t.}\quad p_{\mathbf{N}}^{(i)}+p_{\mathbf{Ne}}^{(i)}+p_{\mathbf{P}}^{(i)}=1$$

*I-M-3-4-1 3 3 Classification model Single Sentences Output

The whole comment coefficient that it will output is:

$$\mathbf{P}_{\text{tri}}=\mathbf{W}^{\top}\mathbf{P}_{\text{sent}}=[w_1\quad w_2\quad\cdots\quad w_n]\begin{bmatrix}p_{\mathbf{N}}^{(1)} & p_{\mathbf{Ne}}^{(1)} & p_{\mathbf{P}}^{(1)}\\ p_{\mathbf{N}}^{(2)} & p_{\mathbf{Ne}}^{(2)} & p_{\mathbf{P}}^{(2)}\\ \vdots & \vdots & \vdots\\ p_{\mathbf{N}}^{(n)} & p_{\mathbf{Ne}}^{(n)} & p_{\mathbf{P}}^{(n)}\end{bmatrix}\qquad w_i=\frac{l_i}{L}=\frac{l_i}{\sum_{j=1}^{n}l_j}$$

*Set comment number is n, the length of sentence n_i is l_i, the comment whole length is $L=\sum_{\mathrm{j}=1}^{\mathrm{i}}l_i$

*I-M-3-4-1 4 Comment coefficient output by 3 clasification model output

Next, we implemented domain-specific word bias intervention. Before making the overall score decision, we introduced domain-customized word intervention through regular expression matching:

If a comment matches a positive feature word but has no negative feature words, we forcibly increase the positive probability of the dual model by 0.08, and then re-normalize.

Conversely, if a comment matches a negative feature word but has no positive feature words, we increase the negative probability by 0.08, and then re-normalize.

After obtaining the corrected probability distributions of the two models and their respective Top-1 labels, the system enters the final decision-making stage. This stage does not simply average the results of the two models; instead, it employs a hierarchical decision-making strategy based on different situations to improve the stability and accuracy of sentiment classification.

First, when the highest confidence scores of both models do not exceed 0.35, it indicates that neither model can reliably judge the text. In this case, the system directly outputs "Neutral" to avoid misjudgments under low confidence conditions.

Second, when the two models show significant conflict, the system introduces an LLM model for decision-making. If one model judges the text as "Positive" and the other as "Negative," it means they are completely opposite in polarity. In this case, the LLM model is invoked to re-analyze the original comment, and its return result is used as the final label. Similarly, when the Top-1 labels of the two models are different, and the difference in their highest scores is greater than or equal to 0.25, a secondary judgment by the large language model is triggered.

When none of the above special branches are triggered, the system enters the regular weighted fusion stage. The specific approach involves linearly merging the three sentiment probabilities of the two models according to predefined weights, using the following formula:

$$P(\text{Negative}) = 0.6 \times P_A(\text{Negative}) + 0.4 \times P_B(\text{Negative})$$

*I-M-3-4-1 5 Weighting formula for combining the results of two sentiment models

The neutral and positive categories are processed similarly, and the category with the highest overall score is ultimately selected as the output.

Furthermore, to avoid unreasonable polarity shifts after weighting, the system incorporates a "direction protection" mechanism. If the Top-1 values of neither of the two base models represent a particular polarity, but the fused result does, the result is considered unsupported, and it is discarded before a new category is selected from the remaining classes. This mechanism further enhances the rationality and robustness of the final decision.

Finally, all comments marked as Negative will go through an additional large language model review process; if the large language model re-determines them as Neutral or Positive, the system will update the label accordingly; otherwise, it will leave the Negative label unchanged.

3.4.2 Keyword Extraction

To improve the stability and effectiveness of keyword extraction, the system adopts a hierarchical keyword extraction process. First, the keyBERT model is used for initial keyword extraction. One to two n-gram candidate phrases are extracted from the original text, and the top three results with the

highest scores are returned. The system checks these candidate words sequentially, selecting the first non-empty result as the initial candidate keyword; if the model fails to extract keywords, it reverts to using the original comment text for subsequent processing.

Subsequently, the system performs Chinese word segmentation on the candidate text and uses a stop word filtering function to remove high-frequency function words with no practical meaning, retaining more informative terms. If an anomaly occurs during stop word processing, the original segmentation results are retained as a backup to ensure uninterrupted processing.

Based on this, the system further performs rule cleaning, applying constraint filtering to the segmentation results, retaining only terms with a length between 2 and 8 characters, without spaces, and not consisting solely of symbols or numbers, thereby removing noisy data and forming a cleaner set of candidate words.

If usable terms still exist after cleaning, the system will reassemble these terms and input them into the SnowNLP model for secondary keyword extraction to further refine the core semantics of the text. For the first three keywords returned by SnowNLP, the system will also filter them according to the above rules and return the first word that meets the criteria as the final keyword.

If the secondary extraction fails to yield effective results, the system will use a fallback strategy, directly returning the first word in the candidate word list after rule cleaning; if the list is empty, an empty string will be returned. This achieves good robustness and usability in keyword extraction under complex text conditions.

At this stage, we obtained more reliable sentiment assessment results and more refined core keywords for the text. The next step is to categorize the content of the comments.

3.5 Topic Classification

In this stage, we use LLM Models to classify the topic. We build a LLM pool, and use Multithreading to make it faster. The Models we use are:

Model	Thread Workers
deepseek/deepseek-chat	20
aliyun/deepseek-v3.2-exp	10

aliyun/kimi-k2.5	8
aliyun/MiniMax-M2.5	8
aliyun/deepseek-v3.1	8
aliyun/qwen-turbo-lastest	8
aliyun/qwen-turbo	8
moonshot/kimi-k2-turbo-preview	3

*I-M-3-5 1 Models use in topic classification

Before official deployment, the system conducts a two-stage evaluation of candidate topic classification models to ensure their usability and stability. The first stage is connectivity testing, which verifies the functionality of the interface connections and the validity of the returned results. Models failing this test are eliminated. The second stage is stress testing, which applies concurrent requests to models that passed the connectivity test and uses multiple test comments in a loop to verify their response stability under high load. Only models with a success rate of 80% or higher are included in the final pool of available models.

In terms of concurrent processing architecture, the system encapsulates each available model as an independent TopicModelWorker and assigns it a dedicated thread pool executor. During actual processing, comment tasks are distributed to different worker task queues using a round-robin approach. Each worker continuously pulls tasks from the queue and executes classification requests through a continuously running processing loop. To improve system robustness, if a model fails to process a specific comment, the system automatically switches to the next available model to continue trying; if all models fail to complete the task, the comment is classified as "Other".

In addition, the system incorporates retry and synchronization mechanisms. Each failed call is retried using an exponential backoff approach to minimize the impact of momentary failures or interface fluctuations. All classification results are uniformly written to a shared result dictionary, and thread locks ensure safe concurrent writes. Simultaneously, a task queue confirmation mechanism ensures that all comments are processed before unified aggregation.

During the classification process, the large language model categorizes comments into candidate topic

tags based on preset prompts and returns the classification results and explanations in JSON format. If the returned content cannot be directly parsed into standard JSON, the system uses regular expressions for fallback parsing to improve the fault tolerance of result extraction.

3.6 Merge Sentiment And Topic Results

In this stage, we will further integrate the two sets of results into a comprehensive analysis table, thereby achieving a joint presentation of comment sentiment and comment topic.

This step uses the sentiment analysis results table and the topic classification results table as input. To ensure the quality of the merging, the system will first deduplicate the topic classification results according to the season and content of the comment, retaining the last result in the duplicate records. Subsequently, the sentiment analysis results, as the main body, are merged with the topic classification results using a left join, so that each comment retains its original sentiment analysis information while supplementing it with the corresponding topic category.

3.7 Output Results

After generating the comprehensive analysis table, we will further output two application-level data results: a comment-level results table and a quarterly summary table.

3.7.1 Comment-Level Results Table

The comment-level results table primarily displays the core analytical conclusions of each comment. The system extracts five key fields from the complete analysis results, including the comment's quarter, comment text, keywords, final sentiment tag, and topic category, and organizes them into a more concise results table.

The final sentiment tag uses results verified by a large language model to ensure more robust output conclusions. After this processing, the comment-level results table can directly answer the two core questions of each comment: "What did it say?" and "What is its emotional tendency?", making it suitable for detailed queries, visualization, and subsequent business analysis.

3.7.2 Quarterly Summary Table

Based on the comment-level results, the system further constructs a quarterly summary table to reflect the overall user feedback for each quarter's works. Specifically, the system groups by quarter, counts the number of positive, negative, and neutral comments in each quarter, and calculates the total number of comments based on this. To ensure a consistent result structure, even if a particular sentiment category is absent in a given season, the system will automatically reset the corresponding count to 0.

Subsequently, the system integrates these sentiment statistics with external rating data, adding corresponding Douban and IMDb ratings for each season, along with the differences between the two. The resulting quarterly summary table includes both quantitative indicators from the platform's rating system and sentiment distribution information from user comments, providing a more comprehensive reflection of the overall performance of works across different seasons.

After all these step, we get the result of ADS output, the final output includes each table and its fields are:

Table Name: ads_comment_theme_analysis
season
comment
keyword
sentiment_association
topic_category

*I-M-3-7 1 Final output of ADS comment

Tabe Name: ads_season_overall_evaluation
season_id
df_douban_rating
imdb_rating
rating_difference

positive_comment_cnt
negative_comment_cnt
netral_comment_cnt
total_comment_cnt

*I-M-3-7 2 Final output of ADS season